1 Teori

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

En klass beskriver de metoder och attribut som objekt som skapas av den klassen skall ha.

- ○ Sant ✔
- ○ Falskt

När man ärver från en superklass till en subklass så får superklassen ett extra ansvar att ta hand om den nya subklassen

- ○ Falskt ✔
- ○ Sant

Om klassen StudentBiljett ärver från klassen Biljett så innebär det att alla metoder och attribut från Biljett också finns i StudentBiljett.

- ○ Sant ✔
- ○ Falskt

Ett objekt har värden på alla attribut.

- ○ Falskt
- ○ Sant ✔

Low Coupling beror på att man har för många objekt i systemet.

- ○ Falskt ✔
- ○ Sant

Det går inte att få High Cohesion samtidigt som man använder Polymorphism

- ○ Falskt ✔
- ○ Sant

High Cohesion går ut på att varje klass skall ha så få och så välavgränsade ansvarsområden som möjligt.

- ○ Falskt
- ○ Sant ✔

Low Coupling går ut på att man skall sträva efter att ha så få och så "lösa" associationer som möjligt mellan klasser i ett system.

- ○ Sant ✔
- ○ Falskt

Inkapsling (Encapsulation) innebär att man låser in metoder i en klass så att de inte kan skada andra klasser.

- ○ Sant
- ○ Falskt ✔

Principen "Program to an Inteface, not an Implementation" går ut på att alla klasser måste ärva från ett interface.

- ○ Sant
- ○ Falskt ✔

Totalpoäng: 10

2 Samlingar av Objekt

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

När man skapar en ArrayList i Java måste man ange vilken typ av objekt den skall innehålla.

- ○ Falskt
- ○ Sant ✔

När man skapar en ArrayList i Java måste man ange hur många objekt den skall innehålla.

- ○ Sant
- ○ Falskt ✔

En ArrayList i Java som innehåller heltal (int) är optimerad för att vara särskilt snabb att använda.

- ○ Falskt ✔
- ○ Sant

Om man till exempel skall skriva ut alla element i en ArrayList så måste man veta hur många de är.

- ○ Falskt ✔
- ○ Sant

En Array och en ArrayList är samma sak.

- ○ Falskt ✔
- ○ Sant

En std::vector i C++ får bara innehålla objekt av klasser du skrivit själv.

- ○ Sant
- ○ Falskt ✔

En array i C++ avslutas alltid med ett null-värde.

- ○ Sant
- ○ Falskt ✔

En tvådimensionell array (i Java och C++) implementeras som en array av arrayer.

- ○ Falskt
- ○ Sant ✔

En tvådimensionell array (i C++) innehåller alltid främst rader, och varje rad innehåller sedan en array med kolumner.

- ○ Falskt ✔
- ○ Sant

Totalpoäng: 9

3 Ny uppgift

Betrakta följande programkod:

```
#include <iostream>

int main(void) {
  int x = 10;

  for (int i=0; i <= x; i++) {
    if (0 == (i / x)) {
      std::cout << "x";
    } else {
      std::cout << i;
    }
  }

  std::cout << std::endl;
}
```

Vad blir utskriften av programmet?

- xxxxxxxxxx10 ✔
- ingenting, det blir kompileringsfel.
- x123456789x
- 123456789x

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Klassen "main" innehåller bara en metod.

- Falskt ✔
- Sant

for-satsen kommer köras tills dess x är noll

- Falskt ✔
- Sant

for-satsen kommer köras tills dess i är noll

- ○ Falskt ✔
- ○ Sant

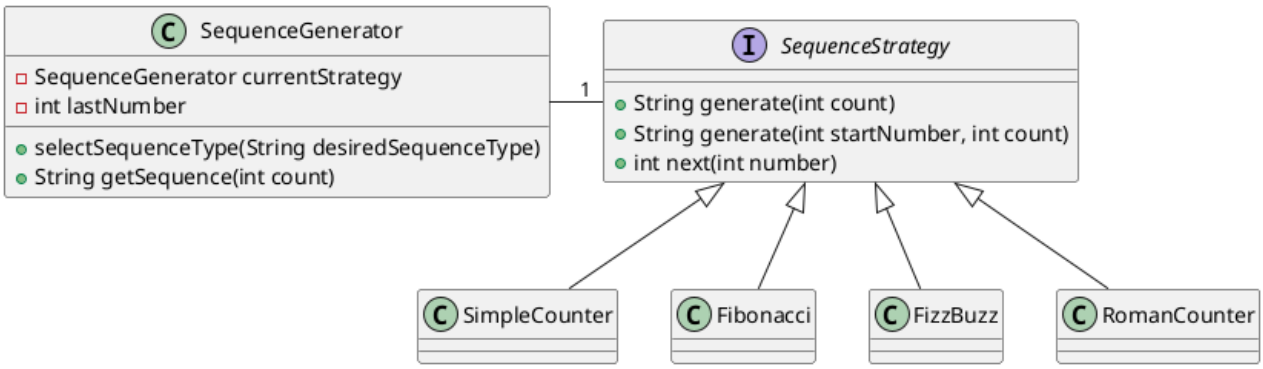
for-satsen kommer köras 11 gånger

- ○ Falskt
- ○ Sant ✔

Totalpoäng: 5

4 Polymorfism och Strategy

Betrakta följande klassdiagram:



Klassdiagrammet visar ett system som kan generera olika typer av nummersekvenser.
Givet diagrammet, markera för varje påstående nedan om programkoden skulle fungera (sant) eller inte fungera (falskt).
(+1 för rätt svar, ingen förändring för fel svar)

i metoden SequenceGenerator::selectSequenceType() hittar man raden "currentStrategy = new SequenceStrategy();"

- ○ Sant
- ○ Falskt ✔

i metoden SequenceGenerator::selectSequenceType() hittar man raden "currentStrategy = new Fibonacci();"

- ○ Sant ✔
- ○ Falskt

i metoden SequenceGenerator::selectSequenceType() hittar man raden "currentStrategy = SequenceStrategy.create(SimpleCounter);"

- ○ Sant
- ○ Falskt ✔

i metoden SequenceGenerator::selectSequenceType() hittar man raden "SequenceStrategy.setStrategy(new SimpleCounter());"

- ○ Sant
- ○ Falskt ✔

i metoden SequenceGenerator::getSequence(int count) hittar man raden "result = currentStrategy.generate(lastNumber, count);"

- ○ Falskt
- ○ Sant ✔

i metoden SequenceStrategy::generate(int count) hittar man raden "while (current < count)"

- ○ Falskt ✔
- ○ Sant

i metoden Fibonacci::generate(int count) hittar man raden "while (current < count)"

- ○ Falskt
- ○ Sant ✔

klassen SimpleCounter implementerar metoden "int SimpleCounter::next(int number)"

- ○ Falskt
- ○ Sant ✔

Totalpoäng: 8

5 Design Patterns

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Strategy pattern handlar om att man skall ha en plan för hur man skall bygga ett system.

- ○ Sant
- ○ Falskt ✔

Designmönstret Strategy pattern använder man för att få high coupling i ett system.

- ○ Falskt ✔
- ○ Sant

Klassen AbstractStrategy deklarerar bara vilka metoder som skall finnas, men implementerar dem inte.

- ○ Sant ✔
- ○ Falskt

Enligt designmönstret Observer Pattern kan klasser med rollen «Observable» anropa klasser som implementerar interfacet Observer.

- ○ Falskt
- ○ Sant ✔

Ett Strategy pattern består av minst tre klasser med rollerna Context, AbstractStrategy, och ConcreteStrategy

- ○ Falskt
- ○ Sant ✔

Designmönstret Decorator använder man när man vill ha lite roligare färger i editorn.

- ○ Falskt ✔
- ○ Sant

Designmönstret Decorator både ärver från en viss basklass och innehåller en referens till ett objekt av basklassen.

- ○ Sant ✔
- ○ Falskt

Designmönstret Adapter använder vi när vi vill ha ett stabilt interface mot något delsystem.

- ○ Falskt
- ○ Sant ✔

Om skapandet av objekt är komplicerat, t.ex. på grund av att vilken klass vi skall använda beror på vissa förutsättningar, så kan vi använda oss av designmönstret Factory Method.

- ○ Sant ✔
- ○ Falskt

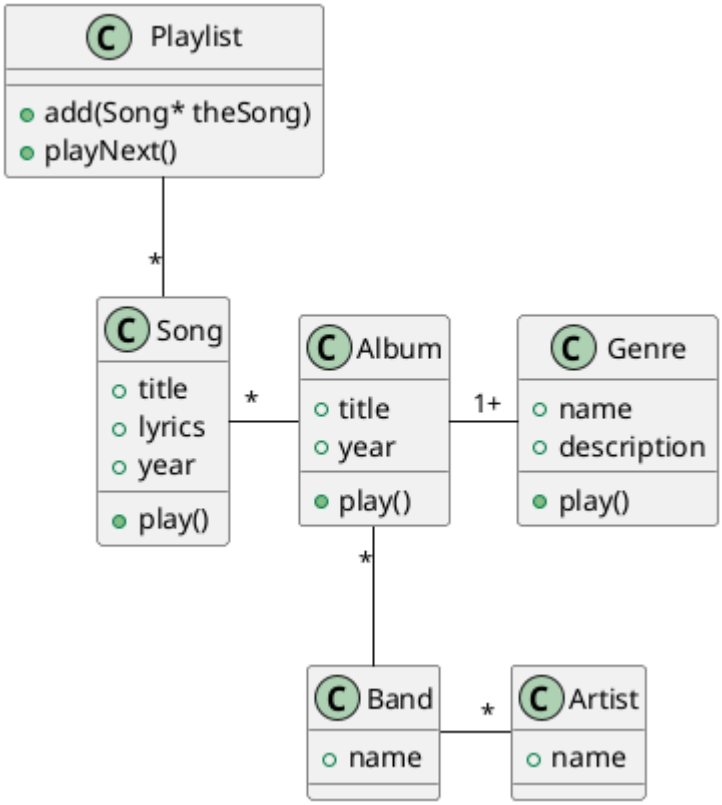
Om skapandet av objekt är komplicerat, t.ex. på grund av att det är många steg som skall utföras med varje nytt objekt, så kan vi använda oss av designmönstret Factory Method.

- ○ Falskt
- ○ Sant ✔

Totalpoäng: 10

6 Klassdiagram

Betrakta följande Klassdiagram:



Klassdiagrammet visar en enkel musiktjänst där man kan bygga upp en spellista av sånger.
För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt).
(+1 för rätt svar, ingen förändring för fel svar)

r1:Album har titeln "A Momentary Lapse of Reason"

- Falskt
- Sant ✔

r1:Album innehåller bland annat en referens till :Song med titeln "Dogs of War"

- Falskt
- Sant ✔

current:Playlist innehåller play[0]:Album, som har titeln "Animals"

- Sant
- Falskt ✔

Objektet :Artist med namnet "Syd Barrett" framför ensam :Song med titeln "Astronomy Dominé".

- Sant
- Falskt ✔

Objektet :Artist med namnet "Roger Waters" har uppträtt med två olika :Band.

- Sant ✔
- Falskt

current:Playlist innehåller play[1]:Song, som har titeln "Have a Cigar".

- Falskt
- Sant ✔

r2:Album kan klassas både som g1:Genre och g2:Genre

- Falskt
- Sant ✔

Man kan direkt lägga till alla Album som klassas som g1:Genre till pl1:Playlist

- Sant
- Falskt ✔

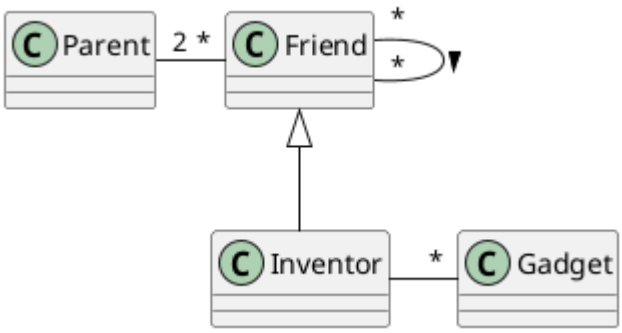
När :Artist "Syd Barrett" lämnade bandet så går det inte längre veta vilka :Album som de sjöng på.

- ○ Falskt ✔
- ○ Sant

Totalpoäng: 9

7 Relationer mellan Klasser

Betrakta följande klassdiagram:



För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt).
(+1 för rätt svar, ingen förändring för fel svar)

Skalman:Inventor har matocksovklocka:Gadget och en skottkärra:Gadget

- Sant ✔
- Falskt

Skalman:Inventor har två vänner, Bamse:Friend, LilleSkutt:Friend

- Sant ✔
- Falskt

Bamse:Friend har en kikare:Gadget

- Sant
- Falskt ✔

LilleSkutt:Friend känner Bamse:Friend

- Falskt
- Sant ✔

Farmor:Parent skapar dunderhonung:Gadget

- ○ Falskt ✔
- ○ Sant

Farmor:Parent är farmor åt Nalle-Maja:Friend

- ○ Falskt ✔
- ○ Sant

LilleSkutt:Friend känner bara Skalman som en Friend, inte som en Inventor.

- ○ Sant ✔
- ○ Falskt

Farmor:Parent är inte en Friend.

- ○ Falskt
- ○ Sant ✔

Skalman:Inventor har en far:Parent och en mor:Parent.

- ○ Sant ✔
- ○ Falskt

Totalpoäng: 9

Betygsgränser

Betygsgränserna för den här tentan är:

Betyg	Procent	Poäng
MAX	100%	60
A	90%	54
B	80%	48
C	70%	42
D	65%	39
E	60%	36

Lycka till!